

129.1. La fonction nombrePanneau() — un nombre dans le panneau

`#include <nombrePanneau>` : `nombrePanneau(colonne, ligne, valeur)` écrit un nombre dans le panneau, comme `nombre()` sur le fond.

1. Ce qu'il faut comprendre

Ce qu'elle fait : `nombrePanneau(colonne, ligne, valeur)` écrit un nombre calculé **dans le panneau** : le score qui reste en place quand le décor défile.

La ligne à écrire : `#include <nombrePanneau>`.

Ses arguments : comme `nombre()`, avec le nombre de chiffres en quatrième, facultatif.

Ce qu'elle coûte : environ **320 octets** la première fois.

Essaie : fais monter le score toutes les secondes, et réécris-le.

On retrouve `nombrePanneau` dans 1 leçon

[130](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **529 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	53 o	← la console, à l'allumage	– la mise en route de la console
main()	43 o	← démarrage	– ta fonction
VBlank	8 o	← la console, à chaque image	– compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	– attend l'image suivante
AttendreVBlank	16 o	← démarrage	– attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre	– attend de pouvoir écrire une case
EcrireTexte	10 o	← textePanneau()	– écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombrePanneau()	– écrit un nombre en chiffres
CopierTuiles	67 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
Tuiles	240 o	← CopierTuiles	– 15 lettres sur 43
"SCORE"	5 o	← textePanneau()	– un texte gravé

Lettres gravées : CEORS0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

panneau()	× 1	15 o sur place, dans main()
textePanneau()	× 1	11 o sur place, dans main() + EcrireTexte (10 o)
nombrePanneau()	× 1	10 o sur place, dans main() + EcrireNombre (33 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <nombrePanneau> : un nombre dans le panneau ----  
  
#include <textePanneau>  
#include <nombrePanneau>    // nombrePanneau() : une valeur, dans le panneau  
#include <panneau>
```

2. UNE VARIABLE — SCORE

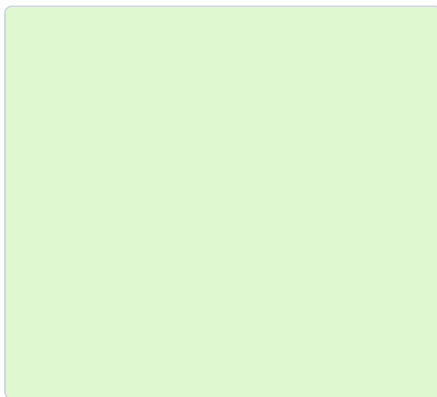
```
uint8_t score = 120;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

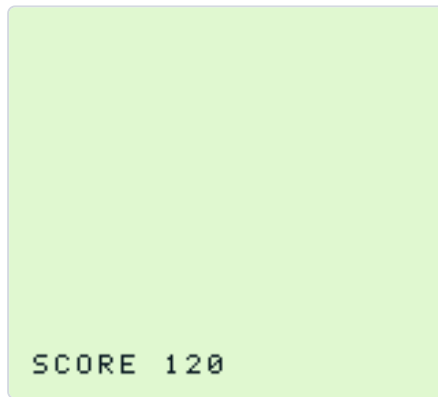
```
int main() {  
    textePanneau(1, 0, "SCORE");  
    nombrePanneau(7, 0, score);    // 120, juste après SCORE  
    panneau(0, 128);  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

SCORE 120 dans la bande du bas.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ trois chiffres sont écrits après SCORE

✓ 1, 2 et 0 : trois chiffres différents

Ensuite : **130. Le score, et le panneau** — chapitre 23 : Un vrai jeu. · gameboy3 — écrire une cartouche Game Boy en C++